

Balaji CRM

An internal CRM for a small real-estate brokerage. This document covers the problem it solves, the solution as built, the full technology stack, the system architecture, and the deployment topology it actually runs on.

CONTENTS

- 01 Problem statement
- 02 The solution
- 03 Technology stack
- 04 System architecture
- 05 End-to-end data flows
- 06 Data model
- 07 Roles, permissions & security
- 08 Deployment stack
- 09 Operating costs & known limits

PRIMARY SURFACE

Mobile-first PWA

INVENTORY SOURCE

WhatsApp groups, read automatically

LIVE ROLES

Owner · Agent · Cold Caller

CORE STACK

Next.js · FastAPI · PostgreSQL

01 Problem statement

A small real-estate brokerage — an owner plus a staff of closing agents and a cold-calling team — currently runs entirely on WhatsApp, phone calls, and scattered notes. That produces three concrete, named problems.

PROBLEM 1

No unified inventory

Property listings are scattered across hundreds of WhatsApp broker groups. Agents cannot quickly find matching inventory for a client's budget and location — they scroll chat history and rely on memory.

PROBLEM 2

No owner visibility

The owner has no way to know which agent is showing which property to which client, or whether a lead is actually being followed up. Oversight depends on staff volunteering the information.

PROBLEM 3

Data leakage risk

There is no access control or audit trail. An agent can walk away with the firm's entire client list — a real, stated risk when staff leave to join a competitor or freelance on the side.

The realistic threat is not an external hacker. It is an insider with legitimate login access trying to extract the client/lead list. Every access-control decision in this system is designed against that specific threat, not a generic one — see §7.

02 The solution

Balaji CRM answers each of the three problems with a structural mechanism, not a bolted-on feature — meaning the fix is enforced in the query layer or middleware, so it cannot be forgotten by some future screen.

→ PROBLEM 1

Automated WhatsApp inventory feed

A gateway service reads the firm's monitored WhatsApp groups directly. An LLM extracts structured fields (location, building, BHK, price, rent vs. sale) from free-text posts, de-duplicates reposts of the same flat, and writes one unified, searchable, filterable inventory — with the original message retained for provenance.

→ PROBLEM 2

Structural activity logging

Every call, site visit, and property-shown-to-client event is its own row, not a mutable field. The owner's live activity feed reads directly from that same table, so "what the owner sees" is always in sync with "what actually happened," with no separate reporting pipeline to drift out of date.

→ PROBLEM 3

Role-scoped access + immutable audit log

Role-based access control is enforced at the query layer (an agent's query is scoped to their own leads at the SQL level, not filtered in the UI). Bulk export exists only for Owner/Manager, every export is itself logged, and the audit log is structural middleware — not something an endpoint has to remember to call.

What is actually built

CORE CRM

- Role-scoped contacts/leads, with masked phone numbers until a qualifying interaction is logged
- Property inventory: manual entries + WhatsApp-sourced listings in one searchable, filterable view
- Site-visit / showing tracking (“who showed what to whom, when”)
- One-tap call logging with outcome + temperature + notes
- Owner activity feed, escalation inbox, team performance view
- Immutable, append-only audit log with an audit viewer for the Owner
- Lead import (bulk CSV/XLSX) with dedup against existing contacts
- Web-push notifications (follow-ups, escalations)

WHATSAPP INVENTORY PIPELINE

- Node/Baileys gateway holds a real WhatsApp Web session and forwards group messages to the backend over a shared-secret webhook
- Messages are journalled raw before parsing, so a prompt fix can be replayed over history without re-reading WhatsApp
- Groq-hosted LLM extraction with a JSON schema, cheap pre-filtering of obvious non-listing chatter, and multi-model rate-limit rotation
- Fuzzy dedup merges reposts of the same flat into one listing with multiple sightings, rather than duplicating rows
- A pairing flow inside the app itself (QR code, group picker) — no terminal access needed once the gateway process is running

03 Technology stack

LAYER	CHOICE	WHY
Frontend	Next.js 16 (App Router) · React 19 · TypeScript · Tailwind CSS 4	One codebase, mobile-first, installable as a PWA, no App Store review cycle
Backend API	FastAPI 0.115 · Python 3.12 · Pydantic 2 · SQLAlchemy 2.0 · Alembic	Async-friendly, fast to build REST APIs, good fit for background extraction jobs
Database	PostgreSQL	Relational integrity for leads/deals/roles; JSONB where fields are genuinely flexible (audit detail, raw WhatsApp message)
Auth	JWT (python-jose) + bcrypt (passlib) · httpOnly cookie sessions	Role claim embedded in token; sliding-expiry sessions with an absolute cap
WhatsApp gateway	Node 22 · @whiskeysockets/baileys 6.7 (WhatsApp Web, multi-device)	No official WhatsApp Business API can read group messages — this is the only way to see broker-group traffic
LLM extraction	Groq API (`groq` SDK) · JSON-schema structured output	High throughput, low cost for a copy-out-of-messy-text task, not a reasoning task; models rotate under per-model rate limits
Dedup	rapidfuzz	Fuzzy string matching for contacts and property listings
Notifications	Web Push (pywebpush + VAPID)	Native to the PWA approach, no separate mobile push service needed
Spreadsheet import/export	openpyxl	Bulk lead import, Owner/Manager exports
Testing	pytest (backend, 100+ tests) · tsc + `next build` (frontend)	Backend logic (RBAC, dedup, extraction) is where correctness risk concentrates

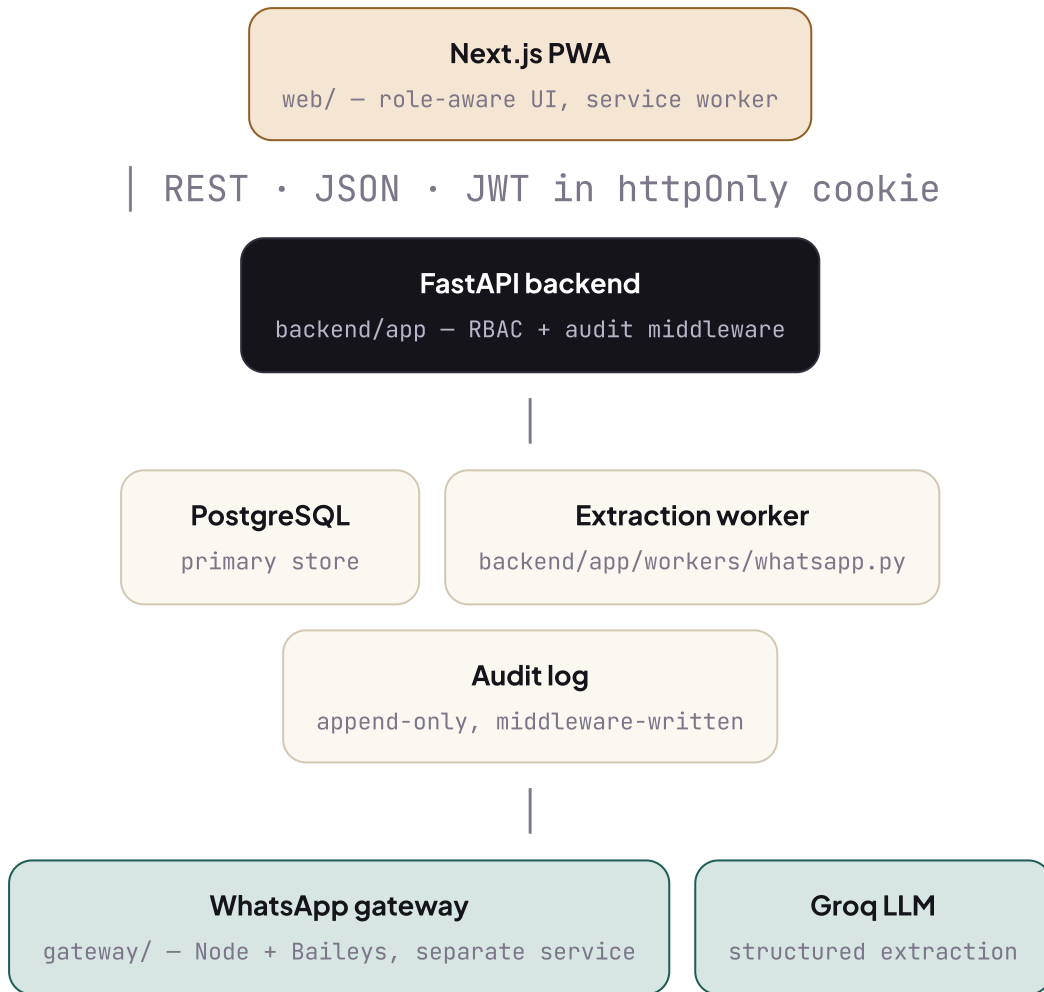
WHY A PWA, NOT A NATIVE APP

One codebase instead of two; faster iteration with no App Store review gate; service workers give good-enough offline viewing of recently-loaded leads/properties. If the team ever needs deep native capability (background location, offline writes), the FastAPI backend and data model don't need to change — only the frontend layer would be rebuilt.

WHY GROQ FOR EXTRACTION

A busy brokerage generates thousands of messages a day, and the task is structured copying out of messy text, not reasoning — it doesn't need a frontier model. The trade-off: Groq has no prompt caching, so the system prompt is repaid on every call, and free-tier rate limits are tight enough to require multi-model rotation (see §8).

04 System architecture



Every arrow is an HTTP call carrying a shared secret, not a shared process — the gateway and the extraction worker can fail independently without taking the core CRM down.

Component notes

FRONTEND — NEXT.JS PWA

Role-aware routing: Owner, Agent, and Cold Caller each land on a different home surface. The browser never calls FastAPI directly — every request goes through a Next.js API route (`/api/crm/[...path]`) that attaches the session JWT from an `httpOnly` cookie, so the API base URL is never exposed to client code.

BACKEND — FASTAPI

RBAC middleware (`app/rbac.py` , `app/scoping.py`) resolves the caller's role on every request and scopes queries accordingly. **Audit middleware** (`app/audit.py`) wraps every read/write on sensitive resources. Neither is opt-in per endpoint.

DATABASE — POSTGRESQL

Soft delete everywhere (`deleted_at`), never hard delete — the audit trail must survive record deletion. Every interaction (call, visit, remark) is its own row in `call_logs` / `activities` , not a mutable field on the lead.

WHATSAPP GATEWAY — ISOLATED BY DESIGN

Kept as a separate Node service, deliberately outside the main FastAPI app: it is the one component built on an unofficial integration (WhatsApp Web, not the Business API), and isolating it limits the blast radius if the session drops or WhatsApp rate-limits it — it cannot take down the core CRM.

05 End-to-end data flows

WhatsApp listing → searchable inventory

1 Message posted

In a monitored WhatsApp group.

2 Gateway relays it

Raw text + metadata forwarded to `/internal/whatsapp/ingest`, authenticated with a shared secret.

3 Raw message stored

Persisted before parsing, so a prompt fix can be replayed over history.

4 LLM extraction

Groq turns free text into structured fields: location, building, BHK, price, rent vs. sale.

5 Dedup check

Fuzzy-matched against existing `properties` rows by building + BHK + price.

6 Inventory updated

New listing inserted, or an existing one gains another sighting — queryable immediately by any staff with inventory-view permission.

Cold call → owner visibility

1 Call placed

Cold caller works their queue, logs outcome via the remark form.

2 Call log written

`POST /calls` — its own row, not a field update.

3 Audit entry written

By middleware, automatically.

4

Follow-up or escalation

A qualifying outcome auto-creates a follow-up task; a flagged call lands in the owner's escalation inbox with a push notification.

5

Owner sees it live

The owner's activity feed reads the same table — no separate reporting pipeline to fall out of sync.

06 Data model

Core tables, in the terms the product uses. Full DDL lives in `docs/DATA_MODEL.md`.

TABLE	PURPOSE	STRUCTURAL NOTE
users	Staff accounts, role, manager hierarchy	Role drives every RBAC scoping decision
contacts	Leads/clients	<code>phone_masked</code> hides full number until a qualifying interaction is logged
properties	Inventory, manual or WhatsApp-sourced	<code>source</code> + <code>raw_message</code> preserve provenance
property_interests	Who showed what to whom, and when	Composite key on <code>(contact_id, property_id, shown_at)</code> — every showing is its own event
call_logs	Every call, outcome, temperature	Own row per call, never overwrites the lead
activities	Site visits, notes, stage changes	Backs both the audit trail and the owner's live feed
audit_log	Append-only action log	No <code>deleted_at</code> ; UPDATE/DELETE revoked at the DB role level where possible

07 Roles, permissions & security

CAPABILITY	OWNER	AGENT	COLD CALLER
View all contacts firm-wide	Yes	Own only	No
Bulk export contacts	Yes	No	No
View unmasked phone/email	Yes	After first logged interaction	Number of lead being called only
Delete a contact or property	Yes	No	No
View firm-wide activity feed	Yes	Own activity only	Own activity only
View the audit log	Full	No	No
Configure WhatsApp ingestion	Yes	No	No

THREAT MODEL

Not an external hacker — an insider with legitimate login access trying to extract the client/lead list before leaving. Mitigations: mandatory pagination and masking (raises the cost of manual scraping), no export capability for Agent/Cold Caller at the API level (not just hidden in the UI), and prompt session invalidation on deactivation.

AUDIT LOGGING

Structural, via FastAPI middleware wrapping every request that touches contacts, properties, call logs, or an export endpoint — no endpoint has to remember to log. Where the database allows it, UPDATE/DELETE on `audit_log` is revoked from the application's own DB role, so even the app's own credentials cannot rewrite history.

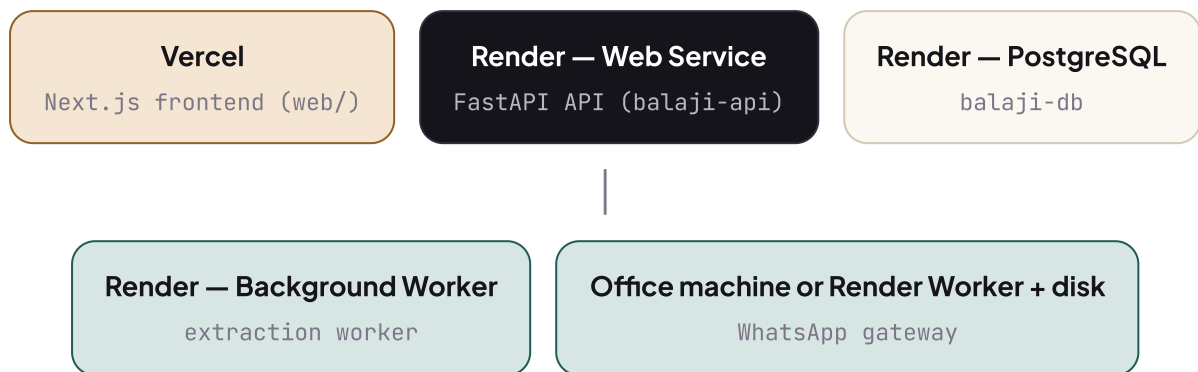
COLD CALLER SCOPE

Sees exactly one surface — their queue — and of the client only a name and mobile number. No lead list, no lead detail, no inventory. Enforced as capabilities the role simply does not hold, so the missing nav items are a consequence of the API refusing the request, not the only control.

SESSIONS

Sliding expiry with an absolute cap: a token renews itself while in use (`SESSION_IDLE_DAYS`, default 30) but can never live past a fixed ceiling from first login (`SESSION_ABSOLUTE_DAYS`, default 90) — so no amount of use makes a session permanent.

08 Deployment stack



HTTPS between Vercel and Render; internal Render network between the API and Postgres; the gateway reaches the API over the public internet with a shared secret.

PIECE	HOST	NOTES
Frontend	Vercel	Root directory set to <code>web</code> ; framework auto-detected as Next.js
API	Render Web Service	Start command runs <code>alembic upgrade head</code> before <code>uvicorn</code> ; health check on <code>/health</code>
Database	Render PostgreSQL	Free tier for 90 days, then ~\$7/mo; internal connection string used by the API
Extraction worker	Render Background Worker	Not available on Render's free plan — <code>EXTRACTION_IN_API=true</code> runs the same loop inside the API as a free-tier fallback
WhatsApp gateway	Office machine (recommended) or Render Worker + persistent disk	Holds the live WhatsApp Web session; needs a persistent disk to survive restarts without re-linking the account

WHY THE GATEWAY IS DEPLOYED SEPARATELY

It drives a real WhatsApp account over an unofficial integration and keeps the linked-device session on disk. Running it on hardware the firm controls keeps that blast radius visible; running it on Render works too, but only with a paid persistent disk — without one, every deploy wipes the session and forces a fresh QR re-link, which is the single most account-flagging thing this process can do.

EXTRACTION THROUGHPUT

Groq's free tier caps at 8,000 tokens/minute and 1,000 requests/day *per model*. `EXTRACTION_MODEL` takes a comma-separated list tried in order, so multiple model names become multiple independent rate-limit buckets on the same key — a model on cooldown is skipped rather than failing the batch.

Configuration surface

<code>render.yaml</code>	Blueprint: database + API + extraction worker (and an optional, commented-out gateway service)
<code>JWT_SECRET</code>	Generated by Render; rotating it invalidates every live session
<code>WHATSAPP_INGEST_SECRET</code>	Shared between the API and the gateway; a mismatch is refused with a 403 that looks identical to a gateway that isn't running
<code>CORS_ORIGINS</code>	Must include the Vercel domain, or every browser request fails as a CORS error
<code>GROQ_API_KEY</code>	Required on both the API and the extraction worker

09 Operating costs & known limits

PIECE	COST
Vercel (Hobby)	Free
Render Postgres	Free for 90 days, then ~\$7/mo
Render Web Service (API)	Free, sleeps after 15 min idle
Render Background Worker	Not free — ~\$7/mo, needed only for the WhatsApp feed
Groq API	Usage-based; free tier is usable but rate-limited, only relevant if the WhatsApp feed is in use

A working CRM without the WhatsApp feed costs nothing beyond the database, after the first 90 days. The feed is what introduces both the recurring worker cost and the operational responsibility of keeping a WhatsApp session alive.

DELIBERATE NON-GOALS

- No native mobile app — PWA only, by design (see §3)
- No payment/commission processing
- No public-facing, buyer-facing listing site — internal tool only
- No legal/document e-signing workflows

WHAT THE SECURITY MODEL DOES NOT ATTEMPT

It cannot stop an agent from memorizing or manually writing down a small number of contacts they're actively working — no software control fully prevents this. The realistic goal is raising the cost and visibility of large-scale extraction, not making leakage physically impossible. Device-level MDM and screenshot prevention are explicitly out of scope for a small-team internal tool.